

S.W.O.R.D. — SIH 2026 Pitch Script

PS: SIH26156 — Universal Log Pre-processing Framework (ULPF)

The biggest mistake would be to **read your PPT**. The slides already show the architecture and tables. Your speech should provide the **story, reasoning, and confidence** behind them.

Your deck currently follows the official SIH structure: **Title → Idea → Technical Approach → Feasibility & Viability → Impact & Benefits → Research & References**.

Below is the script I would use for an internal jury and then refine slightly for the external hackathon.

OPENING — 30–40 sec

[Stand straight. Don't start by reading Slide 1.]

Good morning, respected judges, mentors, and fellow participants.

I am **Divyansh Rajat**, Team Lead and System Architect of **S.W.O.R.D.**

Our team consists of:

Ashish Bhardwaj, Log Processing Engineer;

Sakshi, Normalization and Schema Engineer;

Parampreet Kaur, Backend and Integration Engineer;

Aryan Jeet, Frontend Engineer; and

Saksham, Testing, Documentation and Demo Engineer.

We are presenting **ULPF — Universal Log Pre-processing Framework**, for **Problem Statement SIH26156**.

Our tagline is:

“Different Logs. One Standard. Trusted Everywhere.”

Pause.

Before we explain what ULPF does, let us look at the problem from the perspective of a security team.

SLIDE 1 — THE STORY / PROBLEM — 40–50 sec

Don't spend much time on the title slide.

Imagine a security analyst receives logs from a firewall, a router, a Windows server, a cloud service, and an endpoint security tool.

All of them are describing events happening in the same organization.

But they don't speak the same language.

One source may send **Syslog**.

Another sends **JSON**.

Another uses **CEF or LEEF**.

And a vendor may use a completely proprietary structure.

Even something as simple as a source IP can appear under different field names, structures, or formats.

So before a SIEM, data lake, or ML system can actually use these events consistently, someone has to understand each source and build the required parsing and normalization logic.

That preprocessing layer is the bottleneck we are solving.

Strong transition

The problem is not that enterprises don't have enough logs. The problem is that their logs don't have a common language.

Next slide.

SLIDE 2 — IDEA — 60 sec

Start with the "100 sources" concept

Consider an enterprise with a hundred different log sources.

The conventional approach tends toward source-specific parsers and rules.

So as sources increase, the maintenance burden increases with them.

Point at the diagram:

One hundred sources can become one hundred parser and maintenance problems.

Then introduce your insight:

Our key insight was:

Don't keep building another parser every time a new source appears.

Instead, introduce an adaptive preprocessing layer that separates **source-specific interpretation** from a **stable universal event model**.

Then ULPF

That is ULPF.

ULPF sits between heterogeneous log sources and downstream platforms such as **SIEMs, data lakes and AI/ML systems**.

It ingests the event, detects or discovers its structure, extracts fields, maps them into a common taxonomy, and produces a standardized event.

But there is one principle we do not compromise on:

Normalization must not mean information loss.

Innovation

Therefore, ULPF keeps the original event, the parsed representation, and the normalized representation connected through traceability.

If a vendor sends a field that doesn't fit our universal schema, we don't simply throw it away. We preserve it through extensions and retain the original event.

Five differentiators — don't over-explain

Our differentiation is built around five principles:

Adaptive onboarding.
Lossless normalization.
Bidirectional traceability.
Schema evolution.
And air-gapped deployment.

So our objective isn't simply to parse more formats.

It is to make onboarding a new source a repeatable, controlled process instead of a new development project.

SLIDE 3 — TECHNICAL APPROACH — 75–90 sec

This is your **most important slide**.

Now let me show how ULPF actually works.

Step 1 — Sources

We start with heterogeneous sources — Syslog, JSON, XML, CEF, LEEF, CSV, proprietary formats and future sources.

Step 2 — Ingestion

The ingestion gateway accepts events through file, network or streaming interfaces.

Step 3 — Detection

The format detection layer determines whether the incoming structure is already known.

If it is known, we can use the appropriate parser.

If it is unknown, we don't immediately fail.

Pause.

This is where our adaptive path begins.

Step 4 — Parser Factory

We use template mining, field extraction and semantic mapping to discover the structure of the previously unseen source.

A confidence score determines how certain the mapping is.

Step 5 — Normalization

The extracted fields are then mapped to our common event taxonomy, with **OCSF/ECS-compatible normalization** and extensions for source-specific information.

Step 6 — Traceability

Finally, we attach traceability between the **raw, parsed and normalized events**.

Point to the bottom:

So the output isn't simply:

“Here's a JSON file.”

It is:

“Here's the standardized event, here's where every important value came from, and here's the original event if you need to investigate it.”

DEMO TRANSITION — VERY IMPORTANT

This is where I would stop talking about architecture.

And rather than asking you to believe that this works, we want to show you.

If you have the prototype ready:

In our two-minute prototype demonstration, we start with an **unseen vendor log**.

ULPF discovers its structure, extracts the fields, performs semantic mapping, assigns confidence, and produces a normalized event.

Most importantly, we can trace that normalized event back to the original raw event.

Then show the demo/QR/link.

Your demo narration

Here is our unseen source.

ULPF has detected the structure.

These are the extracted fields.

The system maps them into the common representation.

This is the confidence score.

And now, instead of losing the vendor-specific information, it is retained as an extension.

Finally, this trace ID takes us back to the original event.

Unknown source in — standardized, traceable event out.

That sentence is your **demo punchline**.

TECHNOLOGY STACK — 20–25 sec

Don't read every technology on the slide.

The implementation uses **Python and FastAPI** for the processing and API layers, **Drain3** for template mining, and **Sentence Transformers** for semantic field mapping where required.

PostgreSQL and Redis/Kafka support persistence and processing infrastructure, while **OpenSearch** supports search and analytics.

On the frontend we use **HTML5, CSS3, JavaScript, React and Tailwind CSS**.

The entire system is designed to run through **Docker/Docker Compose**, including in an air-gapped environment.

Important

Don't say:

"AI automatically understands any log."

That's an easy judge attack.

Say:

"We combine deterministic processing with template mining and semantic mapping, with confidence thresholds and human validation for uncertain mappings."

That is much more defensible.

SLIDE 4 — FEASIBILITY & VIABILITY — 50–60 sec

A natural question is:

Can this actually work outside a controlled demo?

We designed ULPF around that question.

Point to the table

For heterogeneous formats, we use modular detection and parsers.

For unknown formats, we use template mining and semantic mapping.

For data preservation, we maintain a raw event vault.

For normalization, we use a common taxonomy with extensible fields.

For schema changes, we maintain versioned mappings and schemas.

Risks

We also don't assume the system will always be correct.

An unknown format can be misinterpreted.

A vendor can introduce new fields.

Log formats can evolve.

Event volume can increase.

And semantic models can make incorrect mappings.

This sentence matters.

So instead of hiding these failure modes, we design for them:

confidence scoring, human review, schema versioning, extension fields, and horizontally scalable workers.

Scalability

At prototype level, our focus is proving the preprocessing workflow and measuring it honestly.

At deployment level, the processing layer can be scaled horizontally by partitioning the event stream across workers.

Strong close

We are not claiming that a student prototype already processes billions of events per day. We are demonstrating the architecture and the measurable path toward that scale.

That line can actually increase credibility.

SLIDE 5 — IMPACT & BENEFITS — 60 sec

Start with the comparison.

Now consider what changes operationally.

Before ULPF

A new source typically means:

study the format → write a parser → test → deploy → maintain.

With ULPF

The intended workflow becomes:

new source → discovery → field mapping → validation → reusable mapping → standardized event.

Then your key impact

This creates five direct benefits.

Operationally, we reduce source-specific parser dependency and accelerate onboarding.

For security, we provide consistent events while preserving the original evidence.

For compliance, transformation provenance remains auditable.

For AI and ML, downstream systems receive more consistent structured events.

And for deployment, ULPF can operate **on-premise, containerized and without mandatory cloud connectivity**.

METRICS — 45 sec

This is where you should sound like engineers, not marketers.

But we don't want to stop at saying that ULPF is "better."

We want to measure exactly how much better.

Point at the table.

Our prototype validation uses seven metrics.

First, **new-source onboarding time** — from the first unseen log to validated normalized events.

Second, **source-specific parser code** required for a new source.

Third, **field mapping accuracy**, validated against known ground truth.

Fourth, **raw-event recoverability**, where our target is one hundred percent.

Fifth, **unknown-field preservation**, also targeted at one hundred percent.

And finally, **processing throughput and normalization latency**, which we will benchmark on defined hardware and workloads.

Important line

The numbers in this table are targets where we haven't yet completed the benchmark; we will replace them with measured results rather than manufacture performance claims.

This is exactly the kind of statement that protects you from a judge asking:

"Where did your 95% come from?"

IMPACT PUNCHLINE

Ultimately, our impact can be summarized in one chain:

Reduce parser development → accelerate source onboarding → preserve forensic evidence → produce analytics-ready events.

Pause.

We are turning heterogeneous logs from a preprocessing burden into a reusable data infrastructure layer.

SLIDE 6 — RESEARCH & REFERENCES — 35–45 sec

Don't read the whole slide.

ULPF is not being designed in isolation.

Our approach builds on established work in log template mining and event standardization.

Drain3 provides the foundation for online template mining.

OCSF provides a vendor-agnostic cybersecurity event model.

ECS provides common event fields for normalization and analytics.

And the SIH26156 problem statement itself defines the core requirements we have designed around — preservation, parsing, normalization, traceability, onboarding, interoperability, AI/ML readiness, air-gapped deployment and containerization.

Then:

We are using these standards and research as foundations — not as substitutes for our own engineering.

FINAL 20–30 SECONDS

Do not end with "Thank you" immediately. Give them the thesis first.

Let us return to the problem we started with.

An enterprise may have hundreds of log sources.

They don't need hundreds of disconnected preprocessing pipelines.

They need a layer that can understand different sources, standardize what can be standardized, preserve what cannot, and maintain the evidence connecting the two.

That is ULPF.

Different Logs. One Standard. Trusted Everywhere.

We are **S.W.O.R.D.**, and this is our solution for **SIH26156**.

Thank you. We are ready for your questions.

The Pitch's Core Story

Your entire presentation should feel like this:

ENTERPRISE



Hundreds of different logs



"Why can't they speak one language?"



Existing approach



Source-specific parsers



Maintenance bottleneck



OUR INSIGHT



Don't build another parser every time



Build an adaptive preprocessing layer



ULPF



Discover → Parse → Map → Normalize → Preserve → Trace



ONE UNIFIED EVENT



SIEM / Data Lake / AI-ML

That's the story.

What Will Make This Pitch Stronger Than a Typical SIH Pitch

1. Don't say "universal" casually

A judge will immediately ask:

"Can you really handle ANY log?"

Your answer:

"We use 'universal' at the framework level, not as a claim that every possible proprietary format is perfectly understood without adaptation. Known formats use parsers; unseen formats go through structure discovery and semantic mapping; unsupported information is retained rather than discarded."

That's much harder to attack.

2. Your strongest moment should be the unknown-source demo

Your existing research specifically identifies the **"learns a new format on stage"** moment as a strong differentiator for SIH26156.

Don't demonstrate:

"Here is a JSON file we prepared yesterday."

Instead:

"This source was not part of our predefined parser set."

Then make the system demonstrate discovery.

That's your **wow moment**.

3. Don't oversell AI

Your strongest technical argument isn't:

"We use Sentence Transformers."

It is:

"We know where automation is safe, where deterministic rules are preferable, and where uncertainty requires human validation."

That makes the architecture look engineered rather than AI-stuffed.

Likely Judge Questions — Your Team Must Be Ready

Judge Question	Your 10–20 sec answer
What is actually novel?	"The novelty is the adaptive preprocessing layer that separates source-specific interpretation from a stable universal event model, while preserving raw data and provenance."
Why not just write parsers?	"That works for a small fixed source set, but parser maintenance grows with source diversity and format changes. ULPF aims to reduce that source-specific development dependency."
What happens with an unknown format?	"Template discovery extracts structure, semantic mapping proposes field meanings, confidence scoring evaluates the mapping, and low-confidence cases go through human validation."
What if mapping is wrong?	"We don't blindly accept it. Confidence thresholds and human validation handle uncertain mappings, while the original event remains preserved."
Why OCSF/ECS?	"They provide established common vocabularies so our output is easier to consume downstream rather than inventing an isolated proprietary schema."
What happens to fields you don't understand?	"They are preserved through extensions and remain connected to the raw event."
How is this lossless?	"The original raw event is retained and linked to the normalized representation through traceability."
Can it handle billions of events?	"Our prototype will establish measured throughput; the production architecture uses partitioned streams and horizontally scalable workers. We don't claim unmeasured billion-event throughput."
Why do you need AI?	"AI is used selectively for semantic mapping of ambiguous fields; deterministic parsing and rules handle cases where they are more reliable."
Why can't Elastic/Splunk already do this?	"Existing platforms provide extensive ingestion and normalization capabilities. Our focus is specifically on an adaptive, extensible preprocessing layer designed around unseen-source onboarding, lossless preservation and provenance, particularly for constrained or air-gapped environments."

That last answer is especially important. **Do not claim that ULPF has invented log normalization.** A judge who knows SIEM infrastructure will immediately challenge you.

Your defensible position is **the combination and implementation of adaptive onboarding + lossless preservation + provenance + extensible schema + air-gapped deployment**, not "we are the first system to normalize logs."

Speaking Distribution for S.W.O.R.D.

I would **not** give every member equal speaking time merely for fairness.

For a competitive pitch:

Divyansh

Opening + Slides 1–2 + transitions + final answer
~45%

Ashish

Technical architecture / parsing
~15%

Sakshi

Normalization / schema / traceability
~10%

Parampreet

Backend / deployment / feasibility
~10%

Aryan

Dashboard / visualization / demo UI
~5–10%

Saksham

Demo operation / testing evidence
~5–10%

If your internal jury gives only a few minutes, **you should do almost the entire pitch** and have members handle only the sections where their expertise adds credibility. Equal speaking time is not the objective; **judge confidence is**.

The supplied SIH research also emphasizes that judges reward a working demo, defensible novelty, realistic feasibility and quantified impact rather than a large feature list.

One final rule

Never memorize this word-for-word.

Memorize these **eight beats**:

Different logs → parser bottleneck → our insight → ULPF → unknown-source demo → lossless traceability → measurable impact → trusted standard.

If you can speak those eight beats naturally, you will sound like the team that **built the system**, rather than students reciting a presentation.

1. What are Logs?

Easy answer

Log ek record hota hai jo batata hai ki kisi system/device/application mein kya event hua.

For example:

```
10:30:21
User: Rahul
Action: Login
IP: 192.168.1.10
Result: Success
```

It tells us:

What happened, when it happened, where it happened, and sometimes who/what caused it.

Examples

- Firewall: connection allowed/blocked
- Server: service started/stopped
- Database: user accessed a table
- Application: error occurred
- Cloud: API call happened
- Endpoint: file/process executed

ULPF connection

ULPF's job is to take these logs from different sources and convert them into a common, usable representation.

2. What are heterogeneous logs? Explain formats.

Heterogeneous = different types/structures of data coming from different sources.

Imagine:

Syslog

```
Aug 26 10:30:21 FW01 ACCEPT src=10.0.0.5 dst=192.168.1.10
```

Human-readable text format.

JSON

```
{
  "src_ip": "10.0.0.5",
  "dst_ip": "192.168.1.10",
  "action": "allow"
}
```

Structured key-value format.

XML

```
<event>
  <src>10.0.0.5</src>
  <dst>192.168.1.10</dst>
  <action>allow</action>
</event>
```

Hierarchical tag-based format.

CSV

```
time,src_ip,dst_ip,action  
10:30,10.0.0.5,192.168.1.10,allow
```

Table/column-based format.

CEF

Common Event Format, commonly used for security events:

```
CEF:0|Vendor|Firewall|1.0|100|Connection|5|src=10.0.0.5 dst=192.168.1.10
```

LEEF

IBM's Log Event Extended Format, designed for event information that can be consumed by QRadar.

Proprietary format

A vendor can create its **own format**, for example:

```
FW01|10.0.0.5|192.168.1.10|ALLOW|TCP|443
```

No universal structure is guaranteed.

Judge one-liner

Heterogeneous logs mean logs that differ in format, structure, field names, vocabulary, and sometimes semantics even when they describe similar events.

3. What is Ingest and Ingestion Gateway?

Ingest

Ingest means data ko system ke andar receive/bring karna.

Example:

```
Firewall  
↓  
Network  
↓  
ULPF
```

Firewall se log ULPF mein aa gaya = **log ingested**.

Ingestion Gateway

It is the **entry point** through which logs enter ULPF.

Think of it like an airport entrance:

```
Different Sources  
↓  
INGESTION GATEWAY  
↓  
ULPF Processing
```

Types

1. File-based

Read logs from:

```
.log  
.csv  
.json
```

2. TCP

Logs arrive through a TCP connection.

Reliable delivery.

3. UDP

Fast but doesn't guarantee delivery.

Commonly used for Syslog.

4. HTTP/HTTPS

Applications send logs through APIs.

```
POST /logs
```

5. Streaming / Message Broker

For high-volume systems:

```
Kafka  
Redis Streams  
etc.
```

Easy judge answer

Ingestion is receiving data; the ingestion gateway is the controlled entry layer that accepts logs through file, network, API, or streaming interfaces.

4. What is a Parser?

A parser takes an unstructured/semi-structured log and breaks it into meaningful fields.

Input:

```
FW01 ACCEPT src=10.0.0.5 dst=192.168.1.10 port=443
```

Parser produces:

```
device = FW01  
action = ACCEPT  
src_ip = 10.0.0.5  
dst_ip = 192.168.1.10  
port = 443
```

Simple analogy

Parser = **translator that breaks a sentence into meaningful pieces.**

ULPF

Raw Log

↓

Parser



Fields

5. What are Downstream Platforms?

Downstream means jo system ULPF ke baad data consume karega.

Logs



ULPF



Downstream Platforms

Examples:

- SIEM
- Data Lake
- AI/ML pipeline
- Threat-hunting system
- Analytics platform
- Dashboard

Easy analogy

ULPF = **water purification plant**

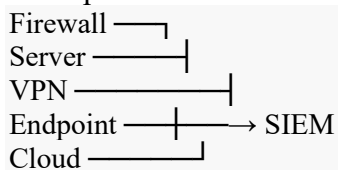
Downstream systems = **people/factories consuming the clean water**

6. What are SIEMs?

SIEM = Security Information and Event Management.

It collects and analyzes security events from many sources.

Example:



SIEM helps security teams:

- collect logs
- search events
- correlate events
- detect suspicious activity
- investigate incidents
- generate alerts

Examples:

- Splunk Enterprise Security
- IBM QRadar SIEM
- Elastic Security

ULPF connection

ULPF prepares heterogeneous events **before they reach downstream security analytics**.

Judge answer

SIEM is the security analysis platform; ULPF is the preprocessing layer that makes heterogeneous events easier and more consistent for such platforms.

7. What are Data Lakes?

A Data Lake is a large centralized storage system where organizations keep large amounts of data, often in raw and processed forms.

Example:

Firewall logs
Server logs
Cloud logs
Application logs
Database logs
↓
DATA LAKE

Unlike a small database designed around a strict schema, a data lake can store large volumes of diverse data.

Why ULPF?

Instead of sending completely inconsistent events:

Different formats
Different field names
Different structures

ULPF can produce:

Consistent + searchable + traceable events

while still preserving the raw event.

8. AI/ML Systems in this context?

AI/ML systems can use normalized logs for:

- anomaly detection
- threat detection
- classification
- forecasting
- behavior analysis
- clustering
- correlation

Example:

Logs
↓
ULPF
↓
Normalized Events
↓
ML Model

↓
Anomaly Score

Why normalization matters

If one source says:

src_ip

and another says:

sourceAddress

and another says:

client_ip

ML pipelines have to deal with inconsistent representations.

ULPF can map them to a common concept such as:

source.ip

Important

Don't say:

"ULPF itself is an AI system."

Your architecture is primarily a **log preprocessing framework**, with AI/ML used selectively for semantic mapping or downstream analytics.

9. What is Normalization?

Normalization means different representations ko common structure/meaning mein convert karna.

Example:

Firewall A:

src_ip = 10.0.0.5

Firewall B:

sourceAddress = 10.0.0.5

Router C:

client_ip = 10.0.0.5

ULPF:

source.ip = 10.0.0.5

Simple analogy

Three people say:

"Paani"

"Water"

"Aqua"

Normalization identifies the common concept:

Water

10. What is Standardization?

Standardization means deciding a common agreed structure, field names, types and vocabulary that everyone follows.

Example:

Without standardization:

```
src_ip  
sourceAddress  
client_ip  
source
```

Standardized:

```
source.ip
```

Difference

This is a common judge question.

Normalization

Existing data ko common form mein convert karna

Transformation

"Is data ko kaise convert karein?"

Standardization

Common form/schema define karna

Rule/standard

"Sabko kis format mein hona chahiye?"

One-liner

Standardization defines the common language; normalization converts different languages into that common language.

11. What is Semantic Mapping?

Semantic mapping means field ka actual meaning samajhkar usko correct common field se map karna.

Example:

```
src_ip  
sourceAddress  
clientIP  
origin_ip
```

All may mean:

```
Source IP
```

So:

```
clientIP → source.ip
```

This is not merely string matching.

Example

```
"decision": "deny"
```

Could map to:

```
event.action = deny
```

because we understand its **meaning**.

ULPF

This is where Sentence Transformers or similar semantic models can assist.

12. What is Template Mining?

This is important for your project.

Suppose ULPF receives:

User Alice logged in from 10.0.0.5

User Bob logged in from 10.0.0.8

User Charlie logged in from 10.0.0.9

The system can discover the common template:

User <*> logged in from <*>

The variable parts are:

Alice

Bob

Charlie

10.0.0.5

10.0.0.8

10.0.0.9

So:

Template mining = repeated log patterns discover karna aur variable parts identify karna.

Drain3 does this online for streams of log messages, extracting templates/clusters and parameters. ([GitHub][2])

13. What is Adaptive Onboarding?

This is one of your most important concepts.

Traditional:

New Source

↓

Developer studies format

↓

Developer writes parser

↓

Testing

↓

Deployment

Adaptive ULPF:

New Source

↓

Detect structure

↓

Discover template

↓

Extract fields

↓

Semantic mapping

↓

Confidence score

↓

Human validation if needed

↓
Reusable mapping

Easy definition

Adaptive onboarding means ULPF can learn the structure of a new log source and create/validate its mapping with minimal source-specific coding.

Don't say

"Any unknown log is automatically understood perfectly."

That's not defensible.

14. Lossless Normalization / Preservation

Lossless means normalization ke baad original information lost nahi hoti.

Example:

Original:

```
FW01|ALLOW|10.0.0.5|192.168.1.10|CUSTOM_X=ABC
```

Normalized:

```
{
  "event.action": "allow",
  "source.ip": "10.0.0.5",
  "destination.ip": "192.168.1.10",
  "extensions": {
    "CUSTOM_X": "ABC"
  }
}
```

And we still store:

```
original_raw_event
```

Why?

For:

- forensic investigation
- compliance
- debugging
- future reprocessing

Judge answer

We don't replace the original event with the normalized event; we preserve the original and create a normalized representation alongside it.

15. What is Provenance?

Provenance means data ki history/source ka record.

Basically:

"Ye value aayi kahan se?"

Example:

source.ip = 10.0.0.5

↑

Raw Event #92831

↑

Firewall-X

↑

Received at 10:30:21

Provenance can record:

- source
- timestamp
- transformation
- mapping used
- schema version
- processing stage

Simple analogy

Courier tracking:

Delhi

↓

Ludhiana

↓

Chandigarh

↓

Delivered

That's provenance.

16. What is an Extensible Schema?

A schema defines:

What fields does our event have and what do they mean?

But vendors may have unique fields.

So instead of:

Universal Schema

= fixed forever

we design:

Core Schema

+

Extensions

Example:

```
{
  "source": {
    "ip": "10.0.0.5"
  },
  "event": {
    "action": "deny"
  },
  "vendor_extension": {
    "policy_id": "P991",
    "zone": "DMZ"
  }
}
```

```
}  
}
```

Why?

New vendors can introduce new information **without breaking the core schema**.

OCSF itself is explicitly designed as an extensible, vendor-agnostic framework. ([Open Cybersecurity Schema Framework][3])

17. What is Bidirectional Traceability?

This sounds complicated but is simple.

It means:

Normalized → Raw

Normalized Event



Trace ID



Original Raw Event

AND ideally:

Raw → Normalized

Original Raw Event



Trace ID



Normalized Event

So you can move **in both directions**.

Example

RAW-82931



TRACE-ABC123



NORMALIZED-ABC123

Why?

If an analyst sees:

source.ip = 10.0.0.5

they can ask:

"Where did this value come from?"

ULPF can take them back to the exact original event.

18. What is Schema Evolution?

Schema evolution means:

Schema ko time ke saath safely update karna without breaking existing data.

Example:

Version 1

source.ip
destination.ip
action

Later:

source.ip
destination.ip
action
user
device

We now have:

Schema v1
Schema v2

Why important?

Log sources change.

New vendors appear.

New fields appear.

Cybersecurity requirements change.

So your schema cannot be permanently frozen.

ULPF approach

Schema Registry

```
graph LR
  SR[Schema Registry] --- v1[v1]
  SR --- v2[v2]
  SR --- v3[v3]
  SR --- Ext[Extensions]
```

19. What is Air-Gapped Deployment?

Air-gapped = system completely isolated from the Internet/external network.

Example:

Internet

X
X
X

Government / Defence Network

↓
ULPF

No mandatory:

- Cloud API
- Internet connection
- external AI API
- SaaS dependency

Why important for your PS?

Security-sensitive organizations may not allow logs to leave their controlled environment.

Your solution

Docker
+
Local models
+
Local database
+
Local processing
=
Air-gapped ULPF

Judge answer

Air-gapped deployment means ULPF can operate entirely inside an isolated network without depending on the public Internet or mandatory cloud services.

20. PostgreSQL, Redis / Kafka — Why?

Don't say all three do the same thing.

PostgreSQL

Database.

Use for:

- mappings
- schema registry
- configuration
- metadata
- provenance records

Think:

Permanent organized storage.

Redis

Very fast in-memory data store.

Can be used for:

- caching
- temporary state
- queues/streams in some architectures
- fast lookup

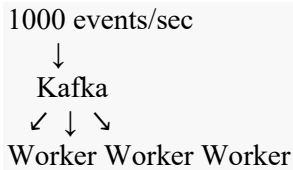
Think:

Fast temporary working memory.

Kafka

Distributed event streaming platform.

Example:



Good for:

- high-volume streams
- buffering
- partitioning
- decoupling producers and consumers

Think:

High-speed railway for events.

Important

For your prototype, **you don't necessarily need PostgreSQL + Redis + Kafka simultaneously.**

If you're asked:

"Why are you using all three?"

A weak answer will hurt you.

Better:

"They represent different architectural options: PostgreSQL for durable metadata, Redis for low-latency state/cache, and Kafka when high-volume streaming and horizontal worker scaling are required. We will use only the components justified by the prototype workload."

21. What is Drain3?

Drain3 is an online log template mining tool based on the Drain algorithm.

Input:

User Alice logged in from 10.0.0.5
User Bob logged in from 10.0.0.8

Drain3 discovers:

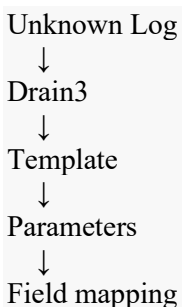
User <*> logged in from <*>

and extracts variable parameters.

It works incrementally on streams and supports persistence, masking and parameter extraction. ([GitHub][2])

Why ULPF uses it?

Because when an unseen log format arrives:



Important judge question

Is Drain3 your innovation?

Answer:

"No. Drain3 is an existing open-source component. Our contribution is how we integrate template discovery with format detection, semantic mapping, confidence validation, normalization, preservation and provenance in the ULPF pipeline."

That answer is much safer.

22. What is OCSF?

OCSF = Open Cybersecurity Schema Framework.

It provides a **vendor-agnostic common schema for cybersecurity events**.

Instead of:

```
Vendor A → source_ip  
Vendor B → srcAddress  
Vendor C → clientIP
```

we can map them into a common representation.

OCSF provides categories, event classes, objects, attributes and extensions. ([Open Cybersecurity Schema Framework][3])

Simple analogy

OCSF = common language for cybersecurity events.

Why ULPF?

ULPF can normalize heterogeneous events toward OCSF-compatible representations.

23. What is ECS?

ECS = Elastic Common Schema.

It is a common set of fields used to normalize event data so it can be searched, analyzed, visualized and correlated consistently within the Elastic ecosystem. ([Elastic][4])

Example:

```
source.ip  
destination.ip  
host.ip  
event.action  
@timestamp
```

OCSF vs ECS

OCSF	ECS
Cybersecurity-focused open framework	Elastic ecosystem common schema
Vendor-neutral	Developed by Elastic
Extensible	Extensible/custom fields possible
Good for security-event normalization	Strong integration with Elastic
ULPF can target it	ULPF can target it

Very important

Don't say:

"OCSF is better than ECS."

Say:

"They solve related normalization problems in different ecosystems. ULPF is designed to be schema-aware and can support mappings such as OCSF and ECS rather than locking itself to one vendor."

24. Existing Platforms — This Is the Most Important Judge Question

You need to know this extremely well.

A judge may say:

"Splunk already does this. Elastic already does this. QRadar already does this. So what is new?"

If you answer:

"They don't support normalization."

you will lose credibility.

They **do**.

A. Splunk Enterprise Security

Splunk Enterprise Security uses the **Common Information Model (CIM)** to normalize data for security applications, and Splunk provides technology-specific add-ons to parse/categorize known data sources. ([Splunk Docs][5])

What it does

Data Sources
↓
Splunk
↓
Parsing / Add-ons
↓
CIM
↓
Security Analytics

Strength

- Huge ecosystem
- Many integrations
- CIM normalization
- Security analytics
- Correlation
- Dashboards

Limitation relevant to ULPF

The important limitation is **not** "Splunk cannot parse logs."

Rather:

Source onboarding commonly depends on technology-specific add-ons/configuration and source-aware normalization.

Splunk's own documentation describes identifying an add-on for each data source and using those add-ons to parse and categorize known sources for CIM compliance. ([Splunk Docs][5])

Your positioning

"We are not trying to replace Splunk. ULPF can act as a preprocessing layer before Splunk or another SIEM, focusing specifically on adaptive onboarding, lossless preservation and provenance."

B. Elastic / Elastic Security

Elastic has:

- Ingest pipelines
- Processors
- ECS
- Integrations
- Parsing
- Log routing
- Search
- Security analytics

Elastic's ingest pipelines can extract fields such as @timestamp, log.level, and host.ip, while ECS provides normalized common fields. ([Elastic][1])

Example

Raw Log

↓

Elastic Ingest Pipeline

↓

ECS

↓

Elasticsearch

↓

Kibana / Security

Strength

Very powerful and flexible.

Limitation relevant to your problem

For arbitrary/unseen sources, parsing generally still requires appropriate pipeline/configuration logic. Elastic's own documentation shows that plaintext logs require parsing based on the log format and that different applications may require different settings. ([Elastic][6])

Your positioning

"Elastic already provides excellent ingestion and normalization. Our differentiation is an adaptive preprocessing layer aimed at reducing the source-specific effort for previously unseen formats, while preserving raw data and transformation provenance."

C. IBM QRadar

QRadar is actually **the platform you need to understand best**, because it is extremely close to your PS.

QRadar uses **Device Support Modules (DSMs)** to identify, parse and normalize log events. IBM documents support for hundreds of DSMs and multiple ingestion protocols. ([IBM][7])

It also has:

- log source autodetection
- DSMs
- custom DSMs
- DSM Editor
- event normalization
- protocols such as Syslog, Kafka, HTTP, etc.

This is important

QRadar even has gateway log sources and autodetection mechanisms.

But IBM's documentation explicitly says that correct parsing depends on an appropriate DSM; unexpected formats may require a custom DSM. ([IBM][8])

For unsupported/custom sources, QRadar provides a DSM Editor and log-source extensions, which can involve configuring properties and regular expressions. ([IBM][9])

So your comparison is:

Capability	QRadar	ULPF
Log ingestion	Yes	Yes
Parsing	Yes, DSM-based	Yes
Normalization	Yes	Yes
Standard taxonomy	Yes	OCSF/ECS-oriented
Known-source support	Strong	Prototype scope
Auto detection	Yes	Yes
Unknown/custom source	DSM/custom configuration may be required	Adaptive discovery + mapping is core focus
Raw event handling	Raw events are collected	Explicit raw-event vault + traceability is core design
Provenance	Some event/source metadata	Field-level/raw↔normalized provenance is core design
SIEM	Yes — it is the SIEM	No — preprocessing layer
Air-gapped/on-prem	Yes	Designed for it
Vendor independence	Platform ecosystem	Framework-level objective

The crucial distinction

QRadar is a SIEM.

ULPF is a preprocessing framework.

That distinction should be central to your pitch.

Your Real Innovation / Novelty

Don't claim:

"We invented log parsing."

Don't claim:

"We invented normalization."

Don't claim:

"Existing SIEMs can't process different formats."

All three are vulnerable.

Instead:

Your proposed innovation is the combination:

HETEROGENEOUS LOG



Adaptive Discovery



Template Mining



Semantic Mapping



Confidence Validation



Common Normalization



Raw Event Preserved |



Provenance / Trace



SIEM / Data Lake / AI

Your strongest differentiation

1. Adaptive onboarding

Focus on reducing manual source-specific work.

2. Lossless normalization

Don't force normalization at the cost of information loss.

3. Traceability

Normalized values remain connected to their source evidence.

4. Extensible schema

Don't break the common model when vendors add unique fields.

5. Air-gapped deployment

No mandatory cloud/Internet dependency.

The 30-Second Answer to "Why ULPF?"

Memorize this:

"Existing platforms like Splunk, Elastic and QRadar already provide powerful log ingestion, parsing and normalization. We are not claiming to replace them. Our focus is the preprocessing bottleneck before analytics: when a new or previously unseen source appears, ULPF aims to discover its structure, map its fields, normalize it into a common schema, preserve the original event and maintain provenance. This makes ULPF a vendor-agnostic preprocessing layer that can feed SIEMs, data lakes and AI/ML systems, including in air-gapped environments."

That's a much more defensible answer than saying **"our system is universal while existing systems are not."**

One final distinction to memorize

PARSER

"What pieces are in this log?"

NORMALIZATION

"Convert those pieces into a common representation."

STANDARDIZATION

"What common representation should everyone use?"

SEMANTIC MAPPING

"What does this field actually mean?"

TEMPLATE MINING

"What recurring structure does this log follow?"

PROVENANCE

"Where did this data/value come from?"

TRACEABILITY

"Can I move between the normalized data and its source?"

ULPF

"Can we automate and connect this whole preprocessing workflow?"

That is the conceptual foundation of your entire SIH26156 solution.

1. What exactly happens when the format is completely unknown?

Easy answer

We don't immediately try to force the unknown log into a predefined parser. We first discover its structure.

The flow is:

```
Unknown Log
↓
Format / Structure Detection
↓
Template Mining
↓
Field Extraction
↓
Semantic Mapping
↓
Confidence Score
↓
Human Validation if needed
↓
Reusable Mapping
↓
Normalized Event
```

Example

Suppose we receive:

```
FW-X|DENY|10.0.0.5|192.168.1.10|TCP|443
```

ULPF doesn't know the vendor.

It observes recurring patterns and identifies something like:

```
Device | Action | Source | Destination | Protocol | Port
```

Then semantic mapping proposes:

```
Action    → event.action
Source     → source.ip
Destination → destination.ip
Protocol   → network.protocol
Port       → destination.port
```

Judge asks:

"What if your system still doesn't understand it?"

Answer:

Then we don't pretend that the mapping is correct. We preserve the original event, retain unmapped fields as extensions, assign low confidence, and send the mapping for human validation.

That answer is much safer than claiming "ULPF understands every format automatically."

2. How do you know the semantic mapping is correct?

This is a **critical question** because semantic mapping is where your AI can make mistakes.

Answer

We don't trust the model blindly. We validate the mapping using multiple signals.

For example:

Field Name

+

Template Context

+

Value Pattern

+

Known Schema

+

Semantic Similarity

↓

Confidence Score

Suppose:

client_ip = 10.0.0.5

The system considers:

- field name: client_ip
- value pattern: IPv4
- context: network connection
- semantic similarity: "client IP" $\approx$ source IP

Then it may propose:

client_ip $\rightarrow$ source_ip

Confidence = 96%

High confidence

Automatically accept or queue for normal validation.

Low confidence

Confidence = 61%

↓

Human Review

Best judge answer

"Semantic mapping is assisted by AI, not blindly controlled by AI. We combine deterministic rules, schema knowledge, value validation and semantic similarity, then use confidence thresholds and human review for uncertain mappings."

3. Why do you need AI if deterministic parsing works?

Don't say:

"AI is better than rules."

That's too broad and usually false.

Better answer:

"We don't use AI where deterministic rules are sufficient. We use it primarily where the problem becomes semantic rather than syntactic."

Example:

Deterministic parser can easily understand:

```
src_ip=10.0.0.5
```

But suppose a vendor uses:

```
originator=10.0.0.5
```

A rule-based parser may know the field only if someone has already configured it.

A semantic model can help determine that:

```
originator
```

in this context may represent:

```
source.ip
```

Architecture principle

Known + deterministic



Rules

Unknown / ambiguous



Template Mining



Semantic Mapping



Confidence



Human Review

Strong answer

"AI is not the parser. It is an assistance layer for discovering meaning when predefined deterministic rules are insufficient."

4. How do you guarantee 100% data preservation?

Be careful here.

You should **not** say "AI guarantees it."

And technically, you cannot guarantee 100% preservation merely because you designed for it. You need an implementation and a test proving it.

Correct answer

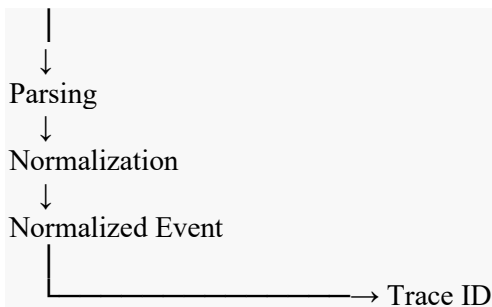
"Our architecture guarantees preservation by design, and we validate that claim through recoverability testing."

The pipeline should be:

RAW EVENT



→ Raw Event Vault



The raw event is **not destroyed or replaced**.

Test

Suppose:

10,000 raw events

After processing:

10,000 trace IDs

You attempt to retrieve all originals.

If:

10,000 / 10,000 recovered

then:

Raw-event recoverability = 100%.

Judge asks:

"What about a field that doesn't fit your schema?"

Answer:

"We preserve it in an extension/custom-field structure and retain the original raw event as the authoritative source."

Important distinction

Say:

"100% is our preservation target and validation criterion."

Don't claim:

"Our prototype guarantees 100%."

until you've actually tested it.

5. How is your approach different from QRadar DSMs?

This is probably one of your **hardest questions**.

QRadar already has:

- Log ingestion
- DSMs
- Parsing
- Normalization
- Auto-detection

- Custom log source support

So don't attack QRadar incorrectly.

Answer

"QRadar DSMs are a mature source-specific mechanism for identifying and parsing supported log sources. ULPF is not trying to replace QRadar as a SIEM or claim that DSMs don't normalize logs."

Then explain your wedge:

QRadar
Source
↓
DSM
↓
Normalized Event
↓
QRadar SIEM

ULPF:
Source
↓
Adaptive Discovery
↓
Template Mining
↓
Semantic Mapping
↓
Normalization
↓
Raw + Provenance
↓
SIEM / Data Lake / AI

Your differentiation

"Our focus is reducing source-specific onboarding dependency for previously unseen formats, while making raw-event preservation and raw-to-normalized provenance first-class parts of the preprocessing layer."

If judge says:

"QRadar supports custom DSMs."

Answer:

"Correct. That's why our claim isn't that QRadar can't handle custom sources. Our research direction is to reduce the manual engineering involved in creating those source-specific mappings, particularly for previously unseen sources, while keeping the preprocessing layer vendor-agnostic and independently deployable."

That's a much stronger answer.

6. How is your approach different from Elastic ingest pipelines?

Again, **don't say Elastic can't do this.**

Elastic supports:

- Ingest pipelines
- Processors
- Parsing
- ECS
- Integrations
- Log routing
- Search and analytics

Answer

"Elastic already provides a very capable ingestion and normalization pipeline. The distinction is that Elastic's pipeline is primarily configured around the Elastic ecosystem and specific ingestion requirements, whereas ULPF is designed as a vendor-agnostic preprocessing layer whose output can feed multiple downstream systems."

Then:

Elastic
Raw Log
↓
Ingest Pipeline
↓
ECS
↓
Elasticsearch
↓
Kibana

ULPF:

Raw Log
↓
ULPF
↓
OCSF / ECS / Common Model
↓
SIEM Data Lake AI/ML APIs

Your strongest distinction

"We are separating the preprocessing problem from the analytics platform."

That is the important architectural idea.

7. Why wouldn't an organization simply use Splunk/Elastic/QRadar?

This is probably the **single most dangerous question**.

If your answer is:

"Because ULPF is better."

you lose.

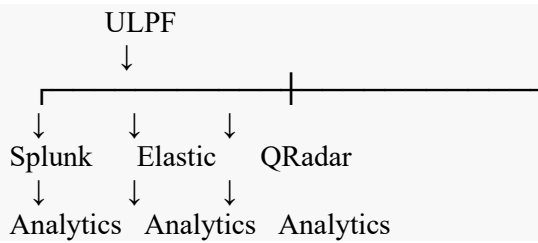
Correct answer

"They absolutely could, and for many organizations they are the right choice. ULPF isn't positioned as a replacement SIEM."

Then:

"Our proposed value is a vendor-agnostic preprocessing layer that can sit before existing SIEMs, data lakes or ML pipelines and focus specifically on heterogeneous-source onboarding, lossless preservation and provenance."

Think:



Why would an organization want this?

Potential reasons:

- Multiple SIEM/data platforms
- Hybrid environments
- Vendor independence
- Air-gapped environments
- Centralized preprocessing
- Reusable mappings
- Forensic/raw-event preservation
- ML pipelines that need consistent data

Best answer

"We don't want the organization to replace its existing SIEM. We want ULPF to make heterogeneous data usable before it reaches whatever downstream platform the organization already uses."

That's the position I'd defend.

8. What happens when your AI mapping is wrong?

Answer

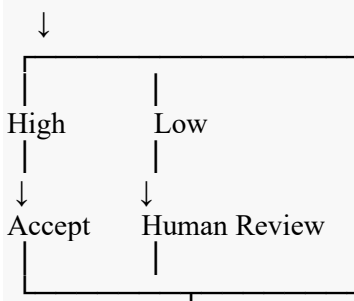
"The system is designed to fail safely rather than silently corrupt the data."

That's a very good phrase to remember.

The flow:

AI Mapping

↓
Confidence Score



↓
Mapping Registry

And throughout:
Original Raw Event

↓
SAFE
PRESERVED

Example

AI says:
originator → source.ip
confidence = 58%

ULPF shouldn't silently accept it.

Instead:
58%
↓
Human Review
↓
Correct Mapping
↓
Mapping Registry

Next time the same source appears, the validated mapping can be reused.

Judge asks:

"So your AI isn't fully autonomous?"

Answer:

"Correct. The objective isn't autonomous guessing; it is reducing manual work while keeping uncertain decisions controllable and auditable."

That is much more credible.

9. How do you measure "universal"?

This is where you need to be precise.

Don't say:

"We support every possible log."

You cannot prove that.

Say:

"We don't measure universality by claiming an unlimited number of formats. We measure it by the diversity of sources, formats and schemas that the framework can process through the same pipeline."

You can create a test matrix:

Dimension	Test Examples
Format	Syslog, JSON, XML, CSV, CEF, LEEF
Source	Firewall, router, server, application

Dimension	Test Examples
Structure	Structured, semi-structured, text
Schema	Known + unknown
Fields	Standard + vendor-specific
Evolution	Schema/template changes
Transport	File, TCP, UDP, HTTP, stream

Then measure:

Coverage

$$Coverage = \frac{\text{Successfully processed test categories}}{\text{Total test categories}} \times 100$$

Adaptation

Measure:

How many unseen sources can be onboarded without writing a new source-specific parser?

Preservation

Measure:

Can every original event be recovered?

Accuracy

Measure:

How accurately are fields mapped?

Strong answer

"For us, universal means extensible across heterogeneous sources, formats and schemas—not magically knowing every possible proprietary format without validation."

That's exactly how I'd frame it.

10. What part is actually your innovation rather than Drain3 + OCSF + ECS?

This is the question you **must answer confidently**.

Because the judge is correct:

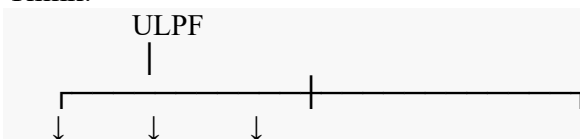
- Drain3 already exists.
- OCSF already exists.
- ECS already exists.

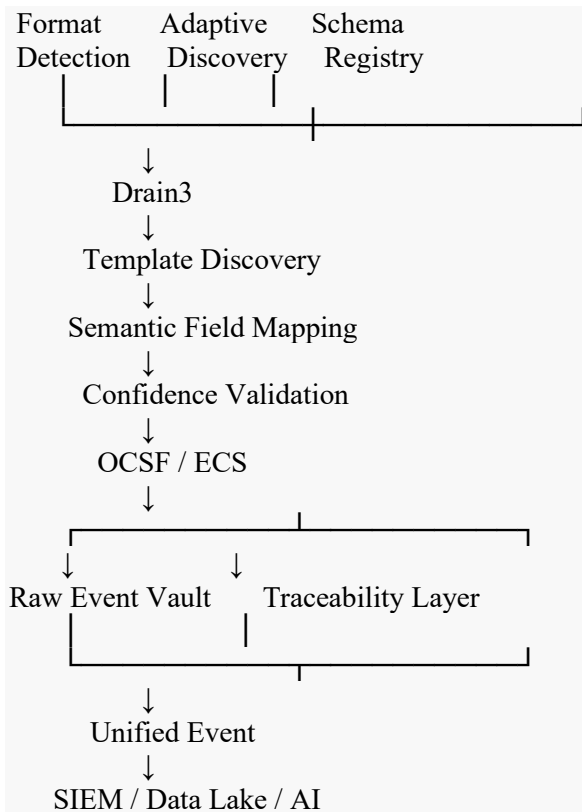
So:

Those components are not your innovation.

Your innovation is the orchestration/framework around them.

Think:





Your actual contribution

1. Adaptive onboarding workflow

Unknown source → discovery → mapping → validation → reusable mapping.

2. Lossless normalization

Normalization doesn't destroy information.

3. Provenance / traceability

Raw
↕
Parsed
↕
Normalized

4. Schema evolution

Core schema + extensions + versioning.

5. Vendor-independent deployment

ULPF isn't tied to a particular SIEM.

6. Air-gapped architecture

No mandatory cloud dependency.

7. Evaluation framework

You're not just building a parser.

You're measuring:

- onboarding time
- parser effort
- mapping accuracy
- raw-event recoverability
- unknown-field preservation
- throughput
- latency

The 20-Second Answer to Question 10

Memorize this:

"Drain3, OCSF and ECS are building blocks, not our innovation. Drain3 handles template mining, while OCSF and ECS provide established schema foundations. Our contribution is the ULPF orchestration layer that connects adaptive source discovery, semantic mapping, confidence-based validation, lossless preservation, provenance, schema evolution and vendor-independent deployment into one preprocessing workflow. The value is in how these components work together to reduce source-specific onboarding effort without sacrificing traceability or original evidence."

Your 10 Answers in One Cheat Sheet

Before the jury, remember these phrases:

Question	One-line answer
1. Unknown format?	Discover → extract → map → confidence → validate → normalize.
2. Mapping correctness?	Rules + value validation + semantic similarity + confidence + human review.
3. Why AI?	Use AI for semantic ambiguity, not where deterministic rules are sufficient.
4. Data preservation?	Preserve raw event separately and validate 100% recoverability.
5. QRadar?	We don't replace DSMs; we target adaptive onboarding + provenance as a vendor-agnostic preprocessing layer.
6. Elastic?	We separate preprocessing from the analytics ecosystem and can feed multiple downstream platforms.
7. Why not SIEM directly?	ULPF can sit before existing SIEMs rather than replace them.
8. AI wrong?	Confidence threshold → human review; never silently corrupt the original.
9. Universal?	Measure diversity + unseen-source onboarding + accuracy + preservation, not "every format ever."
10. Innovation?	The integration of adaptive onboarding, lossless normalization, provenance, schema evolution and vendor-independent deployment.

The one sentence I want your entire team to remember

"ULPF is not trying to reinvent log parsing or replace SIEMs; it is trying to make heterogeneous log preprocessing adaptive, lossless, traceable and vendor-independent."

That is the **defensible core of your SIH idea.**